



Universidad Politécnica de Querétaro

Alumno: Joan Osvaldo Higuera Gallardo

Grupo: S201

30 de Julio del 2026

Práctica No.21: Navigation Stack Update & Delete

Objetivo

Implementar la navegación Stack y completar las operaciones Update y Delete del proyecto Usuarios, manteniendo comunicación con la API y la base de datos.

Descripción de la práctica

La práctica se desarrolló en cuatro partes: agregar Ver detalle a cada usuario, abrir la pantalla Detalles Usuario mediante Stack, precargar y actualizar la información en un formulario, y controlar la eliminación mediante un modal de confirmación.

Desarrollo paso a paso

1. Se agregó la acción Ver detalle a cada elemento mostrado por FlatList.
2. El botón envía el id del usuario seleccionado a la ruta /detalle-usuario.
3. DetalleUsuarioScreen recibe el id y obtiene el registro correspondiente desde la API.
4. La pantalla de detalle muestra el nombre, la edad y las acciones disponibles.
5. El botón Actualizar abre otro elemento de la Stack y conserva el id del usuario.
6. ActualizarUsuarioScreen precarga nombre y edad en el formulario.
7. Antes de guardar se validan los campos y se ejecuta la petición PUT.
8. El botón Eliminar abre un modal sin modificar inmediatamente la base de datos.
9. La petición DELETE se ejecuta únicamente al confirmar Sí, eliminar.
10. Se comprobaron las pantallas de detalle, actualización y eliminación en la aplicación.

Archivos modificados

Los cambios principales se encuentran en ConsultaUsuariosScreen.js, DetalleUsuarioScreen.js, ActualizarUsuarioScreen.js, services/usuariosApi.js y las rutas detalle-usuario.js y actualizar-usuario.js.

Las evidencias siguientes relacionan cada bloque de código con el resultado visible en la aplicación.

Parte 1. Opción Ver detalle

Captura 1. Importación y estilos utilizados para mostrar Ver detalle.

1. Importación de Pressable

```
> Find Aa ab .* No results
1 import React, { useCallback, useState } from 'react';
2 import {
3   ActivityIndicator,
4   FlatList,
5   Pressable,
6   RefreshControl,
7   StyleSheet,
8   Text,
9   View,
10 } from 'react-native';
```

2. Estilo botonDetalle

```
marginRight: 10,
},
botonDetalle: {
  alignItems: 'center',
  flexDirection: 'row',
  paddingHorizontal: 4,
  paddingVertical: 8,
},
```

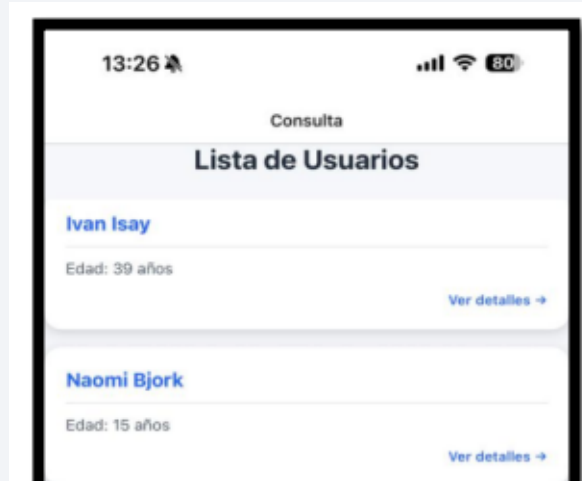
3. Estilo flecha

```
},
flecha: {
  color: '#2457D6',
  fontSize: 24,
  lineHeight: 20,
  marginLeft: 4,
},
```

Pressable habilita la interacción; botonDetalle alinea el texto y la flecha; onPress conserva el id del usuario que debe abrirse.

Parte 1. Evidencia de funcionamiento

Captura 2. Referencia visual de la acción Ver detalles en cada registro.



Captura 3. Lista real de usuarios mostrando Ver detalle.

CONSULTA

Lista de usuarios

1 registro



Joan Prueba

Edad: 23 años

[Ver detalle >](#)

Resultado de la Parte 1

Cada tarjeta cuenta con una acción independiente. Al presionarla, la aplicación conserva item.id y abre la ruta de detalle correspondiente.

Archivo principal: screens/ConsultaUsuariosScreen.js

Parte 2. Pantalla Detalles Usuario

Esta parte recibe el identificador enviado desde Consulta y obtiene únicamente el usuario seleccionado. useLocalSearchParams recupera el id, obtenerUsuario realiza la consulta y useFocusEffect actualiza la información cada vez que la pantalla vuelve a estar activa.

Captura 4. Imágenes agregadas para documentar la recepción, consulta y presentación del usuario.

1. Recepción del identificador

```
} from '../services/usuariosApi';

export default function DetalleUsuarioScreen() {
  const { id } = useLocalSearchParams();
  const router = useRouter();
  const usuarioId = Array.isArray(id) ? id[0] : id;
  const [usuario, setUsuario] = useState(null);
  const [cargando, setCargando] = useState(true);
  const [eliminando, setEliminando] = useState(false);
  const [modalVisible, setModalVisible] = useState(false);
  const [error, setError] = useState('');
```

2. Consulta del usuario en la API

```
26
27 const cargarDetalle = useCallback(async () => {
28   if (!usuarioId) {
29     setError('No se recibió el identificador del usuario.');
```

3. Recarga al enfocar la pantalla

```
45
46 useFocusEffect(
47   useCallback(() => {
48     cargarDetalle();
49   }, [cargarDetalle])
50 );
51
```

4. Presentación de los datos

```
97 <Text style={styles.etiqueta}>INFORMACIÓN</Text>
98 <Text style={styles.titulo}>Detalles del usuario</Text>
99
100 <View style={styles.tarjeta}>
101   <View style={styles.avatar}>
102     <Text style={styles.avatarTexto}>
103       {usuario.nombre.trim().charAt(0).toUpperCase()}
104     </Text>
105   </View>
106
107   <View style={styles.filasDato}>
108     <Text style={styles.label}>Nombre</Text>
109     <Text style={styles.valor}>{usuario.nombre}</Text>
110   </View>
111   <View style={styles.divisor} />
```

Lugares principales: recepción del id y estados (DetalleUsuarioScreen.js, líneas 17-25), cargarDetalle (27-44), useFocusEffect (46-50) y presentación de nombre y edad (94-116).

Parte 2. Evidencia de funcionamiento

Captura 5. Detalles del usuario seleccionado mediante navegación Stack.

Detalle del usuario



Resultado

La pantalla muestra el registro de Joan Osvaldo recibido mediante el parámetro id. Los botones Actualizar y Eliminar permiten continuar con los siguientes elementos de la Stack.

Parte 3. Actualización del usuario

El botón Actualizar abre /actualizar-usuario y envía el mismo id. La nueva pantalla obtiene el registro, precarga nombre y edad, valida los cambios y ejecuta actualizarUsuario mediante una petición PUT.

Captura 6. Código agregado para navegar, precargar el formulario y guardar los cambios.

1. Navegación hacia Actualizar

```
52 const abrirEdición = () => {
53   router.push({
54     pathname: '/actualizar-usuario',
55     params: { id: String(usuario.id) },
56   });
57 };
58
117
118 <Pressable
119   accessibilityRole="button"
120   onPress={abrirEdición}
121   style={({ pressed }) => [
122     styles.botonActualizar,
123     pressed && styles.botonPresionado,
124   ]}
125   <Text style={styles.textoBotonOscuro}>Actualizar</Text>
126 </Pressable>
```

2. Precarga de datos

```
34 export default function ActualizarUsuarioScreen() {
35   const { id } = useLocalSearchParams();
36   const router = useRouter();
37   const usuarios = array.isArray(id) ? id[0] : id;
38   const [nombre, setNombre] = useState('');
39   const [edad, setEdad] = useState('');
40
41   const precargarUsuario = async () => {
42     if (!usuarios) {
43       setError('No se recibió el identificador del usuario.');
```

3. Validación y guardado

```
77 const guardarCambios = async () => {
78   const nombreLimpio = nombre.trim();
79   const edadNumero = Number(edad);
80
81   if (nombreLimpio.length < 3) {
82     setError('El nombre debe tener al menos 3 caracteres.');
```

4. Formulario de actualización

```
128 <View style={styles.targeta}>
129   <Text style={styles.label}>Nombre</Text>
130   <TextInput
131     onChangeText={setNombre}
132     placeholder="Nombre del usuario"
133     style={styles.input}
134     value={nombre}
135   />
136
137   <Text style={styles.label}>Edad</Text>
138   <TextInput
139     onChangeText={setEdad}
140     placeholder="Edad del usuario"
141     style={styles.input}
142     value={edad}
143   />
144
145   <Pressable
146     accessibilityRole="button"
147     disabled={guardando}
148     onPress={guardarCambios}
149     style={({ pressed }) => [
150       styles.boton,
151       pressed && styles.botonPresionado,
152       guardando && styles.botonDeshabilitado,
153     ]}
154   >
155     <Text style={styles.textoBoton}>
156       {guardando ? 'Guardando...' : 'Guardar cambios'}
157     </Text>
158   </Pressable>
```

La actualización se separa en cuatro pasos: navegación desde Detalle, precarga del usuario, validación de los campos y envío de los datos modificados a la API.

Parte 3. Evidencia de funcionamiento

Captura 7. Formulario de actualización con los datos actuales precargados.

← Actualizar usuario

EDICIÓN
Actualizar usuario
Los datos actuales están precargados. Modifica lo necesario y guarda.

Nombre
Joan Osvaldo

Edad
22

Guardar cambios

Resultado

Los campos muestran Joan Osvaldo y 22 antes de editar. Al presionar Guardar cambios se valida la información y se ejecuta PUT /v1/usuarios/{id}.

Parte 4. Eliminación con confirmación

La eliminación no se realiza directamente. El botón Eliminar cambia modalVisible a true; el modal ofrece Cancelar o Sí, eliminar. Solo la confirmación ejecuta eliminarUsuario y su petición DELETE.

Captura 8. Código agregado para mostrar el modal y controlar la petición DELETE.

1. Confirmación y petición DELETE

```
59 const confirmarEliminacion = async () => {  
60   try {  
61     setEliminando(true);  
62     await eliminarUsuario(usuario.id);  
63     setModalVisible(false);  
64     router.replace('/consulta');  
65   } catch (err) {  
66     setError(err.message);  
67     setModalVisible(false);  
68   } finally {  
69     setEliminando(false);  
70   }  
71 }  
  
74 export async function eliminarUsuario(id) {  
75   return solicitar('/v1/usuarios/' + id), {  
76     method: 'DELETE',  
77     headers: {  
78       Authorization: AUTHORIZATION,  
79     },  
80   }  
};
```

2. Botón Eliminar

```
128  
129   <Pressable  
130     accessibilityRole="button"  
131     onPress={() => setModalVisible(true)}  
132     style={({ pressed }) => [  
133       styles.botonEliminar,  
134       pressed && styles.botonPresionado,  
135     ]}  
136   >  
137     <Text style={styles.textoBoton}>Eliminar</Text>  
138   </Pressable>
```

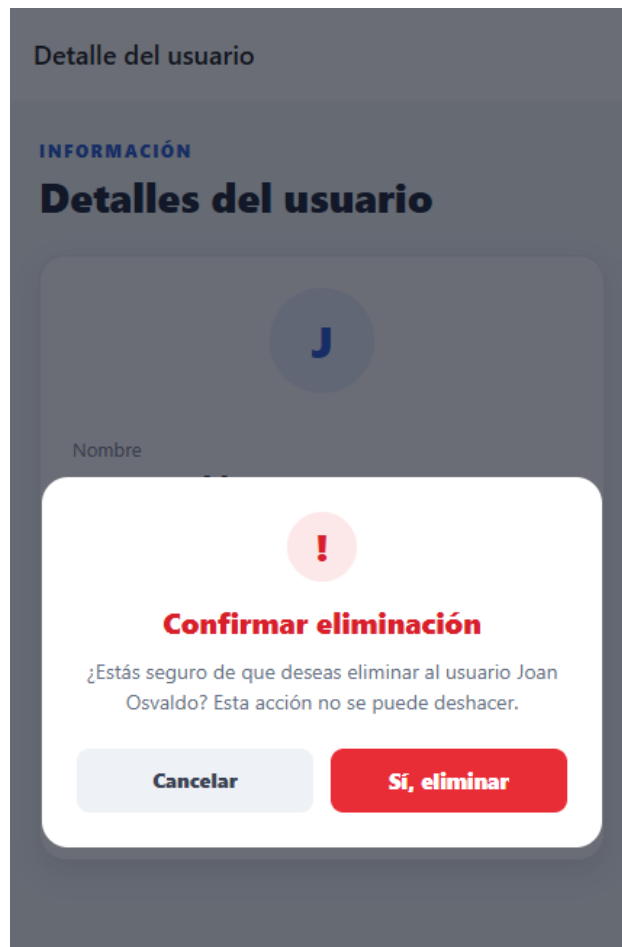
3. Modal y decisión del usuario

```
141  
142   <Modal  
143     animationType="fade"  
144     onRequestClose={() => !eliminando && setModalVisible(false)}  
145     transparent  
146     visible={modalVisible}  
147   >  
148     <View style={styles.fondoModal}>  
149       <View style={styles.modal}>  
150         <View style={styles.iconoAlerta}>  
151           <Text style={styles.iconoAlertaTexto}>!</Text>  
152         </View>  
153         <Text style={styles.modalTitulo}>Confirmar eliminación</Text>  
154         <Text style={styles.modalTexto}>  
155           ¿Estás seguro de que deseas eliminar al usuario {usuario.nombre}?  
156           Esta acción no se puede deshacer.  
157         </Text>  
158  
159         <Pressable  
160           accessibilityRole="button"  
161           disabled={!eliminando}  
162           onPress={confirmarEliminacion}  
163           style={([  
164             styles.confirmar,  
165             eliminando && styles.botonDeshabilitado,  
166           ])}  
167         >  
168           <Text style={styles.textoBoton}>  
169             {eliminando ? 'Eliminando...' : 'Sí, eliminar'}  
170           </Text>  
171         </Pressable>  
172       </View>  
173     </View>
```

Si el usuario cancela, el modal se cierra sin afectar la información. Si confirma, se elimina el registro y la navegación regresa a Consulta.

Parte 4. Evidencia de funcionamiento

Captura 9. Modal mostrado antes de eliminar al usuario Joan Osvaldo.



Conclusión

La Práctica 21 quedó completada con navegación Stack y las operaciones Update y Delete. La aplicación conserva el usuario seleccionado mediante su id, precarga los datos antes de actualizar y protege la eliminación mediante una confirmación explícita.

Proyecto: usuarioApi | API: FastAPI | Aplicación móvil: Expo Router